

โครงงาน

Survive Janggam

จัดทำโดย

6704062610348 นัฐนันท์ชัย รื่นรมย์

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

สารบัญ

บทที1 บทนำ 3

1.2 ที่มาและความสำคัญของโครงการ 3

1.3 ประเภทของโครงการและวัตถุประสงค์ 3

1.4 ประโยชน์ที่คาดว่าจะได้รับ 3

1.5 ขอบเขตของโครงการ 4

1.6 เครื่องมือและโครงสร้างไฟล์ 4

บทที่2

2.1เนื้อเรื่องย่อ **4**

2.2 วิธีเล่น **5**

2.3 class Daigram 5

2.6 Constructor 8

2.7Encapsulation 10

2.8 Composition 10

2.9 Polymorphism 11

2.10 Abstract 11

2.11 Inheritance 12

2.12 GUI 13

2.13 GUI component 14

2.14 Event handling 14

2.15 อัลกอริทึมที่สำคัญในโปรแกรม 17

บทที่3

ปัญหาที่พบ **20**

จุดเด่น และคำแนะนำ **20**

บทที่**1** บทนำ

1.2 ที่มาและความสำคัญของโครงการ

โครงการนี้นำความรู้การเขียนโปรแกรมเชิงวัตถุมาประยุกต์กับเกมยิงซอมบี้มุมมองด้านข้าง ผู้เล่นต้องเคลื่อนที่ เล็งอาวุธ หลบอุปสรรค และจัดการทรัพยากรระหว่างต่อสู้
การทำงานเหล่านี้เกี่ยวข้องกับวัตถุหลายชนิดที่มีสถานะและพฤติกรรมต่างกัน
จึงเหมาะสำหรับศึกษาการออกแบบคลาสและการทำงานร่วมกันของออบเจ็กต์

การพัฒนาเกมช่วยให้เห็นผลของการออกแบบโปรแกรมอย่างเป็นรูปธรรม เช่น การแยกผู้เล่นออกจากอาวุธ การใช้คลาสแม่ร่วมกันสำหรับบอส และการแยกชิ้นส่วนภาพตัวละครเพื่อรองรับการเล็งด้วยเมาส์
เมื่อปรับพฤติกรรมของคลาสใดคลาสหนึ่งจะสามารถติดตามผลที่เกิดกับระบบเกมได้

1.3 ประเภทของโครงการและวัตถุประสงค์

เป็นโครงการพัฒนาซอฟต์แวร์เกมเพื่อความบันเทิงและการศึกษา รูปแบบ Desktop Application
สำหรับผู้เล่นหนึ่งคน ใช้ Java Swing และ Java2D

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1.ผู้พัฒนาได้ฝึกแบ่งความรับผิดชอบของคลาส

2.แก้ปัญหาการทำงานร่วมกันของระบบ และตรวจสอบข้อผิดพลาดจากการเล่นจริง

3.ผู้เล่นได้ฝึกสังเกตจังหวะโจมตี ตัดสินใจใช้เงิน เกราะ และสกิลให้เหมาะสม

4.ผลงานใช้เป็นตัวอย่างสำหรับอธิบาย OOP และเป็นฐานในการพัฒนาเกมเพิ่มเติมได้

1.5 ขอบเขตของโครงการ

ตารางแผนงาน

ลำดับ	รายการ	10 ก.ย – 20 ก.ย	21 ก.ย – 20 พ.ย	21 พ.ย – 24 พ.ย	25 พ.ย – 30 พ.ย
1	จัดทำตัวละครในเกม				
2	ศึกษาข้อมูล				
3	โค้ด				
4	จัดทำเอกสาร				
5	ตรวจสอบ ข้อผิดพลาด				

1.6 เครื่องมือและโครงสร้างไฟล์

เครื่องมือหรือเทคโนโลยี	**หน้าที่**
Java และ JDK	ภาษาและเครื่องมือคอมไพล์โปรแกรม
Visual Studio Code	เขียน แก้ไข และรันโค้ด
Swing	สร้าง JFrame, JPanel, JButton และรับเหตุการณ์
Java2D และ ImageIO	วาดฉาก รูปภาพ อาวุธ และเอฟเฟกต์
Git และ GitHub	จัดเก็บเวอร์ชันซอร์สโค้ดและเผยแพร่โปรเจกต์

บทที่2 ส่วนการพัฒนา

2.1 เนื้อเรื่องย่อ

ผู้เล่นจะรับบทเป็นคนเอาชีวิตรอดโดยต้องจัดการซอมบี้ให้ได้เพื่ออัปเกรดอาวุธและเกราะ

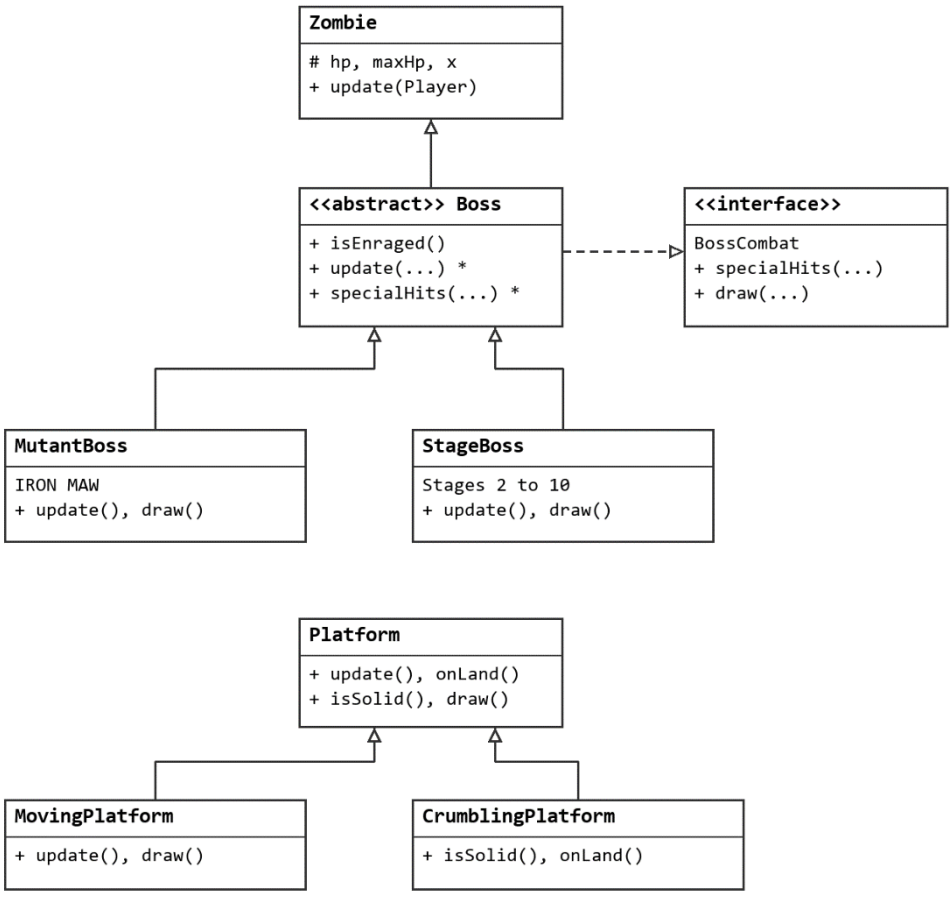
2.2 วิธีเล่น

ปุ่ม	ความสามารถ
A	ทางซ้าย
D	ไปทางขวา
Spacbar	กระโดด
คลิกซ้าย	ยิง
F	ปล่อยพลัง
R และ E	รีโหลด และเก็บอาวุธใกล้ตัว
Shift	Dash ตามทิศที่หัน
1 ถึง 5	เลือกอาวุธที่ปลดล็อกแล้ว
H และ B	ใช้เงิน 100 ฟื้น HP 35 หรือเพิ่มเกราะ 30

เป้าหมายหลัก

ต้องจบ BOSS ให้ได้

2.3 class Diagram



1. คลาส Zombie

เป็นคลาสพื้นฐานของซอมบี้ เก็บข้อมูลพลังชีวิต (hp) พลังชีวิตสูงสุด (maxHp) และตำแหน่งแนวนอน (x) รวมถึงเมธอด update(Player) สำหรับปรับพฤติกรรมของศัตรูตามผู้เล่น

2. คลาสนามธรรม Boss

สืบทอดจาก Zombie เพื่อใช้ข้อมูลและความสามารถพื้นฐานร่วมกัน โดยเพิ่มเมธอด isEnraged() สำหรับตรวจสอบสถานะคลั่ง และกำหนดเมธอดนามธรรม เช่น update() และ specialHits() ให้คลาสบอสลูกกำหนดพฤติกรรมของตนเอง ไม่สามารถสร้างออบเจ็กต์จากคลาส Boss โดยตรงได้

3. อินเทอร์เฟซ BossCombat

กำหนดข้อตกลงเกี่ยวกับการต่อสู้ของบอส ได้แก่ specialHits() สำหรับตรวจสอบการโจมตีพิเศษ และ draw() สำหรับวาดบอส คลาส Boss implements อินเทอร์เฟซนี้

4. คลาส MutantBoss และ StageBoss

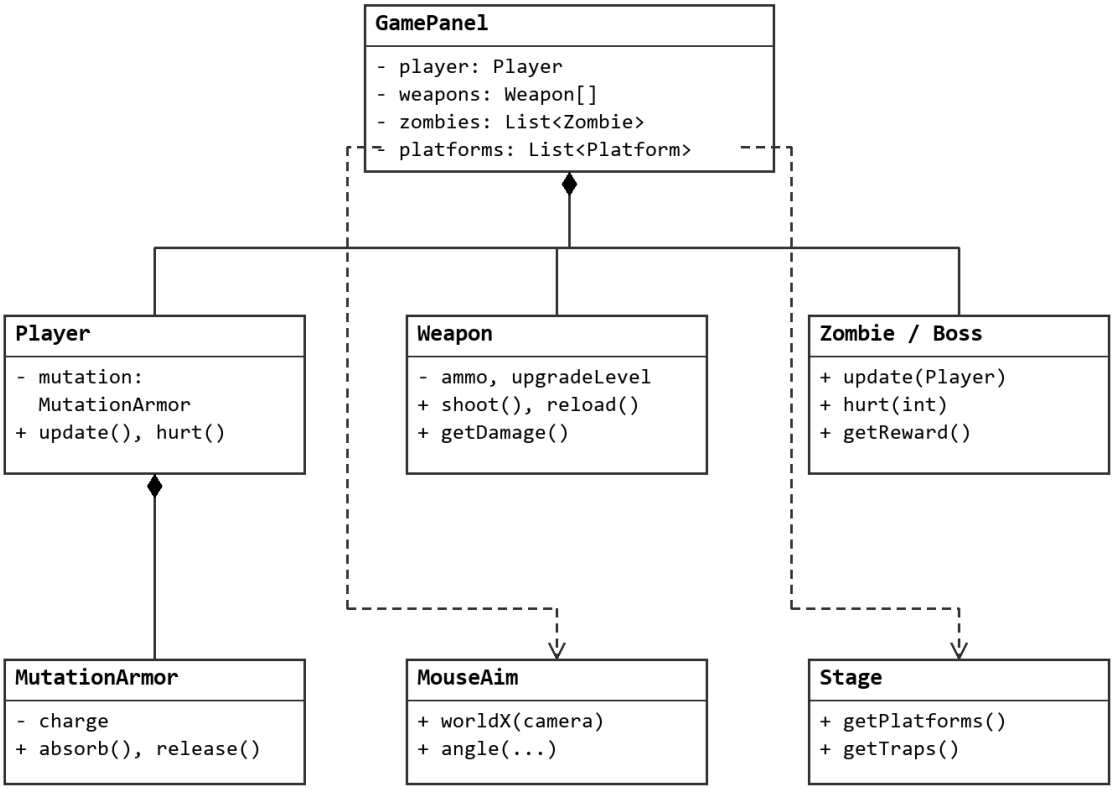
ทั้งสองคลาสสืบทอดจาก Boss โดย MutantBoss ใช้สำหรับบอส IRON MAW ส่วน StageBoss ใช้สำหรับบอสในด่านที่ 2–10 แต่ละคลาสกำหนดการเคลื่อนไหว การโจมตี และการแสดงผลของตนเอง ทำให้เกมเรียกใช้งานผ่านชนิด Boss ร่วมกันได้ตามหลัก Polymorphism

5. คลาส Platform และคลาสลูก

Platform เป็นคลาสพื้นฐานของแท่นยืน มีเมธอดสำหรับอัปเดตแท่น ตรวจการลงเหยียบ ตรวจว่าแท่นรองรับผู้เล่นได้หรือไม่ และวาดภาพ โดยมีคลาสลูก ได้แก่

- MovingPlatform แท่นเคลื่อนที่ ซึ่งปรับการทำงานของ update() และ draw()
- CrumblingPlatform แท่นที่พังเมื่อเหยียบ ซึ่งปรับการทำงานของ onLand() และ isSolid()

2.5 คลาสไดอะแกรมด้านการประกอบวัตถุ



1. คลาส GamePanel

จัดการออบเจ็กต์ที่ใช้ระหว่างเล่น ได้แก่ ผู้เล่น (Player) ชุดอาวุธ (Weapon[]) รายการศัตรู (List<Zombie>) และรายการแท่นยืน (List<Platform>) เพื่อควบคุมการอัปเดต การต่อสู้ และการแสดงผลภายในด่าน

2. คลาส Player

จัดการสถานะและพฤติกรรมของผู้เล่น โดยมี update() สำหรับอัปเดตการเคลื่อนไหว และ hurt() สำหรับรับความเสียหาย ภายในผู้เล่นมีออบเจ็กต์ MutationArmor เป็นส่วนประกอบสำหรับจัดการพลังเกราะชีวภาพ

3. คลาส MutationArmor

เก็บพลังสะสมไว้ในตัวแปร charge มีเมธอด absorb() สำหรับสะสมพลังจากความเสียหายที่เกราะรับ และ release() สำหรับปล่อยพลังเมื่อเข้าเงื่อนไขการใช้สกิล

4. คลาส Weapon

จัดการข้อมูลอาวุธ เช่น กระสุน (ammo) และระดับการอัปเกรด (upgradeLevel) พร้อมเมธอด shoot() สำหรับยิง reload() สำหรับบรรจุกระสุน และ getDamage() สำหรับคำนวณค่าความเสียหายของอาวุธ

5. คลาส Zombie และ Boss

เป็นศัตรูที่ GamePanel จัดการ มีเมธอด update(Player) สำหรับอัปเดตพฤติกรรมตามผู้เล่น hurt(int) สำหรับรับความเสียหาย และ getReward() สำหรับคืนค่ารางวัลเมื่อกำจัดศัตรูได้

6. คลาส MouseAim

ช่วยแปลงตำแหน่งเมาส์บนหน้าจอเป็นตำแหน่งในโลกเกมผ่าน worldX(camera) และคำนวณมุมเล็งผ่าน angle(...) โดยคำนึงถึงตำแหน่งกล้อง

7. คลาส Stage

จัดเตรียมข้อมูลของด่าน เช่น แท่นยืนจาก getPlatforms() และกับดักจาก getTraps() เพื่อให้ GamePanel นำไปใช้สร้างพื้นที่เล่น

2.5 รูปแบบการพัฒนา Application / Applet

โครงงานนี้พัฒนาในรูปแบบ Java Desktop Application โดยใช้ภาษา Java และ Java Swing ในการ สร้างหน้าต่างเกมและส่วนติดต่อผู้ใช้ (GUI) โปรแกรมทำงานบนเครื่องคอมพิวเตอร์ของผู้ใช้ผ่าน Java Runtime Environment โดยมีMain เป็นจุดเริ่มต้นของโปรแกรม และ Game เป็นส่วนหลักในการควบคุมการทำงานของเกม

2.6 Constructor

```
public GamePanel(JFrame var1, int var2) {
    this(var1, var2, false);
}
```

ทำหน้าที่เริ่มต้นหน้าจอเกม โดยรับหน้าต่างเกม **var1** และหมายเลขด่าน **var2**

```
public GamePanel(JFrame var1, int var2, boolean var3) {
    this.mouseAim = new MouseAim();
    this.barrels = new ArrayList();
    this.heldKeys = new HashSet();
    this.player = new Player(100, 350);
    this.playerAnimation = new PlayerAnimation();
    this.bullets = new ArrayList();
    this.zombies = new ArrayList();
    this.zombieAnimations = new IdentityHashMap();
    this.dyingZombies = new ArrayList();
    this.weaponDrops = new ArrayList();
    this.unlockedWeapons = new boolean[]{true, false, false, false, false};
    this.weapons = Weapon.create();
    this.rng = new Random();
    this.weaponImages = new BufferedImage[5];
    this.selected = 0;
    this.spawned = 0;
    this.killed = 0;
    this.credits = 0;
    this.camera = 0;
    this.lastSpawn = 0L;
    this.lastFire = 0L;
    this.banner = "";
    this.bannerUntil = 0L;
    this.lastFireEffect = 0L;
    this.muzzleFlash = new MuzzleFlashEffect();
    this.bossPractice = var3;
    this.weaponImages[0] = this.loadSprite("GUN/gun1.png");
    this.weaponImages[1] = this.loadSprite("GUN/gun2.png");
    this.weaponImages[2] = this.loadSprite("GUN/gun3.png");
    this.weaponImages[3] = this.loadSprite("GUN/gun4.png");
    this.weaponImages[4] = this.loadSprite("GUN/gun5.png");
```

this.player = new Player(100, 350); // สร้างผู้เล่นที่ตำแหน่งเริ่มต้น (100, 350)

this.credits = 0; // กำหนดเงินเริ่มต้นเป็น 0

this.bossPractice = var3; // กำหนดว่าจะเปิดโหมดฝึกบอสหรือไม่ ตามค่าที่ส่งเข้ามาครับ

2.7 Encapsulation

```
public final class MutationArmor {
    public static final int MAX_CHARGE=100,RADIUS=210,DAMAGE=120;
    private int charge,releaseTicks,animationTicks;
    public void update(){animationTicks++;if(releaseTicks>0)releaseTicks--;}
    public void absorb(int blockedDamage,int remainingArmor){
        if(remainingArmor<=0){charge=0;return;}
        if(blockedDamage>0)charge=(int)Math.min(MAX_CHARGE,(long)charge+(long)blockedDamage*3);
    }
    public int getCharge(){return charge;}
    public boolean isReady(){return charge==MAX_CHARGE;}
    public boolean release(){if(!isReady())return false;charge=0;releaseTicks=24;return true;}
    public int getReleaseTicks(){return releaseTicks;}
    public int getAnimationTicks(){return animationTicks;}
    public int getVisualStage(){return releaseTicks>0 || charge==100?2:charge>=40?1:0;}
}
```

คลาส MutationArmor ใช้หลักการ Encapsulation โดยกำหนดตัวแปร charge, releaseTicks และ animationTicks เป็น private เพื่อป้องกันการเข้าถึงข้อมูลโดยตรงจากภายนอกคลาส

2.8 Composition

```
// พลังชีวิตและเกราะ
private int hp = 100;
private int armor = 40;
private final MutationArmor mutation=new MutationArmor();
```

คลาส Player ใช้หลักการ Composition โดยมีออบเจ็กต์ของคลาส MutationArmor เป็นส่วนประกอบภายใน เพื่อจัดการพลังสะสมและการปล่อยพลังของเกราะชีวภาพ ผู้เล่นแต่ละออบเจ็กต์จึงมีสถานะเกราะของตนเอง

2.9 Polymorphism

```
for(Iterator<Zombie> it=zombies.iterator();it.hasNext();){

    Zombie z=it.next();
    if(z.dead() && !z.isDying()){

        killed++;
        credits += z.getReward();

        tryDropWeapon(z);

        z.startDeath();

        ZombieAnimation anim =
            zombieAnimations.computeIfAbsent(
                z,
                key -> new ZombieAnimation()
            );

        anim.playDeath();

        dyingZombies.add(z);

        it.remove();
        continue;
    }

    z.update(player);
    if(z instanceof BossCombat && ((BossCombat)z).specialHits(player)){player.hurt(z.getDamage());playerAnimation.playHurt();}
    ZombieAnimation anim = zombieAnimations.computeIfAbsent(
        z, key -> new ZombieAnimation()
    );
```

ตัวแปร z มีชนิดเป็น Zombie แต่สามารถอ้างอิงออบเจ็กต์ซอมบี้หรือบอสที่สืบทอดจาก Zombie ได้ เมื่อเรียก z.update(player) โปรแกรมจะเลือกใช้เมธอด update() ตามคลาสจริงของออบเจ็กต์

2.10 Abstract

```
public abstract class Boss extends Zombie implements BossCombat {
    protected Boss(int x,String name,int level){super(x,name,level);}
    public final boolean isEnraged(){return hp<maxHp/2;}
    protected final void pursue(Player player){super.update(player);}
    @Override public abstract void update(Player player);
    @Override public abstract boolean specialHits(Player player);
    @Override public abstract void draw(Graphics2D g,int camera,boolean facingRight);
}
```

คลาส Boss ใช้หลักการ Abstract เพื่อเป็นต้นแบบร่วมของบอส โดยรวบรวมความสามารถพื้นฐานและกำหนดเมธอดนามธรรม เช่น update() และ specialHits() ให้คลาสลูกกำหนดการทำงานของตนเอง คลาส MutantBoss และ StageBoss จึงสามารถมีพฤติกรรมการเคลื่อนไหวและโจมตีแตกต่างกันได้ โดยไม่สามารถสร้างออบเจ็กต์จากคลาส Boss โดยตรง

2.11 Inheritance

```
public abstract class Boss extends Zombie implements BossCombat {
    protected Boss(int x,String name,int level){super(x,name,level);}
    public final boolean isEnraged(){return hp<maxHp/2;}
```

คลาส Boss ใช้หลักการ Inheritance โดยสืบทอดจากคลาส Zombie ทำให้บอสใช้ข้อมูลและความสามารถพื้นฐานของซอมบี้ร่วมกันได้ เช่น พลังชีวิต ตำแหน่ง และการรับความเสียหาย พร้อมเพิ่มความสามารถเฉพาะของบอส

2.12 GUI

โปรแกรมพัฒนาส่วนติดต่อผู้ใช้ด้วย Java Swing และวาดภาพเกมด้วย Java AWT โดยมีหน้าจอหลักดังนี้

1.**หน้าจอเมนูหลัก** แสดงชื่อเกม ภาพพื้นหลัง และปุ่มสำหรับเริ่มใช้งานเกม

2.**หน้าจอเลือกด่าน** แสดงปุ่มเลือกด่าน สถานะการปลดล็อก และข้อมูลบอส เพื่อให้ผู้เล่นเลือกด่านที่ต้องการ

3.**หน้าจอเล่นเกม** แสดงผู้เล่น ศัตรู แท่นยืน กับดัก และฉาก พร้อมข้อมูลสถานะ เช่น พลังชีวิต เกราะ พลังเกราะชีวภาพ กระสุน และเงิน

Component ที่ใช้

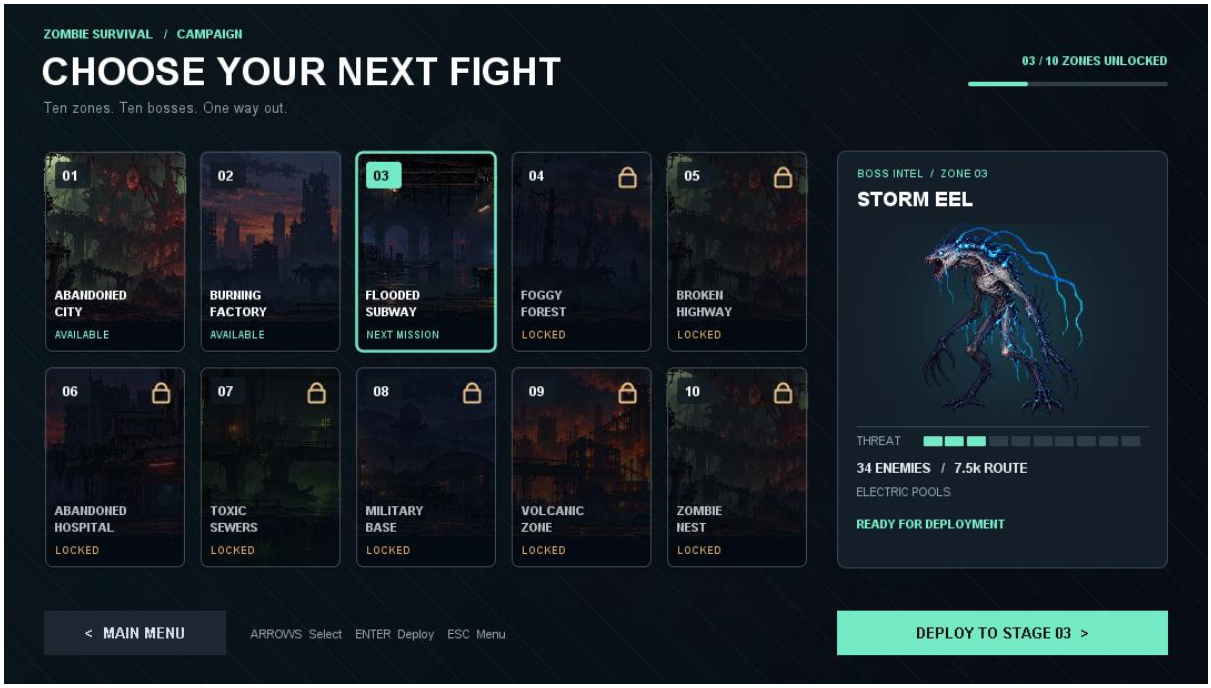
JFrame, JPanel, JButton, Graphics2D

1.หน้าเริ่มเกม



2.

(สร้างจาก StartPanel และปุ่ม START GAME)



หน้าเริ่มเกมใช้ GridBagLayout จัดปุ่มกึ่งกลาง ส่วนหน้าเลือกด่านใช้การ์ด 10 ใบและแผงข้อมูลบอส ด่านล็อกกดดูข้อมูลได้ แต่ปุ่ม Deploy ถูกปิดจนกว่าจะปลดล็อก

3.หน้าการเล่น



หน้าการเล่น เลือด เกราะ พลังกลายพันธุ์ กระสุน และแถบบอส

2.13 โครงสร้างของ GUI component

component	หน้าที่
JFrame	หน้าต่างหลัก เปลี่ยน content pane เพื่อสลับหน้าจอ
JPanel	StartPanel, LevelPanel และ GamePanel ใช้จัดวางและวาดภาพ
JButton	ปุ่มเริ่มเกม กลับเมนู Deploy และ StageCard
GridBagLayout	จัดตำแหน่งปุ่มและแผงย่อยในหน้าเริ่มเกม
StageCard	ปุ่มวาดเอง แสดงฉาก หมายเลข สถานะล็อกและสถานะเลือก
Graphics2D	วาดฉาก ตัวละคร แถบค่า เป้าเมาส์ ร้านค้า และเอฟเฟกต์
Swing Timer	กระตุ้น tick() และ repaint() ตั้งช่วงประมาณ 16 ms
JOptionPane	เลือกหมายเลขด่านใน BossPractice
InputMap และ ActionMap	รองรับลูกศร Enter และ Esc ในหน้าเลือกด่าน

2.14 Event handling

1.start

```
start.addActionListener(e -> {
    frame.setContentPane(new LevelPanel(frame));
    frame.revalidate();
    frame.repaint();
});
```

เมื่อผู้เล่นคลิกปุ่ม START GAME โปรแกรมจะรับเหตุการณ์ผ่าน ActionListener และเปลี่ยนหน้าจอเป็น LevelPanel สำหรับเลือกด่าน จากนั้นเรียก revalidate() เพื่อจัดวางองค์ประกอบใหม่ และ repaint() เพื่อวาดหน้าจอให้แสดงผลล่าสุด

2.คลิกการ์ดด่าน

```
private class StageCard extends JButton {
    private final int level;
    StageCard(int level){
        this.level=level;setContentAreaFilled(false);setBorderPainted(false);setFocusPainte
        getAccessibleContext().setAccessibleName("Stage "+level+" "+Stage.get(level).name+(
        addActionListener(e->select(level));getModel().addChangeListener(e->repaint());
    }
```

มื่อผู้เล่นคลิกการ์ดด่าน ActionListener จะเรียกเมธอด select(level) เพื่อบันทึกหมายเลขด่านที่เลือก ตรวจสอบว่าด่านปลดล็อกแล้วหรือไม่ และปรับสถานะปุ่ม DEPLOY จากนั้นวาดหน้าจอใหม่เพื่อแสดงกรอบการ์ดที่เลือกและข้อมูลบอสของด่านนั้น

3. Deploy หรือ Enter

```
style(launch,primary: true);style(back,primary: false);add(launch);add(back);
launch.addActionListener(e->startSelected());back.addActionListener(e->back());
bind(key: "LEFT",()->select(Math.max(1,selected-1)));bind(key: "RIGHT",()->select(Math.min(10,selected+1)));
bind(key: "UP",()->select(selected>5?selected-5:selected));bind(key: "DOWN",()->select(selected<=5?selected+5:selected));
bind(key: "ENTER",this::startSelected);bind(key: "ESCAPE",this::back);select(selected);
```

```
private void startSelected(){
    if(selected>unlocked || frame==null)
        return;
        frame.setContentPane(new GamePanel(frame,selected));
        frame.revalidate();
        frame.repaint();}
```

เมื่อผู้เล่นคลิกปุ่ม DEPLOY หรือกด Enter โปรแกรมจะเรียก startSelected() เพื่อตรวจสอบว่าด่านที่เลือกปลดล็อกแล้วและมีหน้าต่างเกมพร้อมใช้งาน จากนั้นเปลี่ยนหน้าจอเป็น GamePanel ของด่านที่เลือก

4.กดและปล่อยคีย์

```
KeyboardFocusManager.getCurrentKeyboardFocusManager().addKeyEventDispatcher(e->{
    if(frame.getContentPane() instanceof GamePanel){
        if(e.getID()==KeyEvent.KEY_PRESSED || e.getID()==KeyEvent.KEY_RELEASED){
            ((GamePanel)frame.getContentPane()).handleKey(e.getKeyCode(),e.getID()==KeyEvent.KEY_PRESSED);
            return true;
        }
    }
    return false;
});
```

ใช้ KeyEventDispatcher รับเหตุการณ์จากแป้นพิมพ์ขณะที่แสดงหน้า GamePanel โดยตรวจสอบทั้งการกดปุ่ม (KEY_PRESSED) และการปล่อยปุ่ม (KEY_RELEASED) จากนั้นส่งรหัสปุ่มและสถานะไปยังเมธอด handleKey() เพื่อควบคุมตัวละครและคำสั่งภายในเกม โดยส่งค่า true เมื่อกด และ false เมื่อปล่อยปุ่ม ส่วน return true

5.ขยับหรือกดลากเมาส์

```
MouseAdapter mouse=new MouseAdapter(){
    private void aim(MouseEvent e){if(!shopOpen && !finished){mouseAim.move(e.getX(),e.getY());player.faceTowards(mouseAim.worldX(camera));}}
    @Override public void mouseMoved(MouseEvent e){aim(e);}
    @Override public void mouseDragged(MouseEvent e){aim(e);}
    @Override public void mousePressed(MouseEvent e){aim(e);if(SwingUtilities.isLeftMouseButton(e) && !shopOpen && !finished)fire=true;}
    @Override public void mouseReleased(MouseEvent e){if(SwingUtilities.isLeftMouseButton(e))fire=false;}
};
addMouseListener(mouse);addMouseMotionListener(mouse);
if(bossPractice){zombies.add(BossFactory.create(x: 650,level,stage.worldWidth));bossSpawned=true;message("BOSS PRACTICE: "+stage.getBossName());}
timer=new javax.swing.Timer(16,e->tick());timer.start();
```

รับเหตุการณ์ขยับเมาส์ผ่าน mouseMoved() และการกดปุ่มเมาส์ค้างแล้วลากผ่าน mouseDragged() โดยทั้งสองเมธอดเรียก aim() เพื่อบันทึกตำแหน่งเมาส์และปรับทิศทางการหันของผู้เล่น

6.กดและปล่อยคลิกซ้าย

```
@Override public void mousePressed(MouseEvent e){
    aim(e);
    if(SwingUtilities.isLeftMouseButton(e) && !shopOpen && !finished)
        fire=true;
    }
```

```
    @Override public void mouseReleased(MouseEvent e){
        if(SwingUtilities.isLeftMouseButton(e))
            fire=false;
        }
};
addMouseListener(mouse);addMouseMotionListener(mouse);
```

เมื่อผู้เล่นกดคลิกซ้าย โปรแกรมรับเหตุการณ์ผ่าน mousePressed() เพื่ออัปเดตตำแหน่งเล็ง และกำหนด fire = true เพื่อให้ระบบเกมดำเนินการยิงตามเงื่อนไขของอาวุธ

7. Timer ทำงาน

```
timer=new javax.swing.Timer(16,e->tick());
timer.start();
```

ใช้ javax.swing.Timer กำหนดให้เรียกเมธอด tick() ทุกประมาณ 16 มิลลิวินาที เพื่ออัปเดตสถานะเกม

2.15 อัลกอริทึมที่สำคัญในโปรแกรม

1. อัลกอริทึมอัปเดตเกม (Game Loop)

```
private void tick(){

    Long now=System.currentTimeMillis();

    if(finished || shopOpen){repaint
                                    Platform platform - GamePanel.tick()
                                    Source: ProjectGameOOP_93311ca3
    for(Platform platform:platforms)platform.update();
    for(StageTrap trap:traps)trap.update();
    player.update(left,right,jump,stage.worldWidth,platforms);

    for (SupplyPickup supply : supplies) {
        if (supply.collect(player)) {
            switch (supply.getType()) {
                case HEALTH: message(s: "HEALTH +25"); break;
                case ARMOR: message(s: "ARMOR +25"); break;
                case AMMO: weapons[selected].refillAmmo(); message(s: "AMMO REFILLED"); break;
                case CREDITS: credits += 50; message(s: "CREDITS +50"); break;
            }
        }
    }
```

เมธอด tick() ทำหน้าที่อัปเดตสถานะเกมในแต่ละรอบ โดยตรวจสอบก่อนว่าเกมสิ้นสุดหรือร้านค้าเปิดอยู่หรือไม่ หากเข้าเงื่อนไขจะวาดหน้าจอและหยุดการอัปเดตรอบนั้น หากเกมยังดำเนินอยู่ จะอัปเดตแท่นยืน กับดัก และผู้เล่นตามลำดับ จากนั้นตรวจสอบการเก็บไอเทมและดำเนินการตามประเภท เช่น ฟื้นฟูพลังชีวิต เพิ่มเกราะ เติมกระสุน หรือเพิ่มเงิน

2.กระโดดและลงบนแท่น

```
double previousBottom = y + height;
vy += 0.64;
double nextY = y + vy;
ground = false;
support = null;
if (vy >= 0) {
    double landingY = 470;
    Platform landingPlatform = null;
    for (Platform platform : platforms) {
        if (platform.isSolid() && platform.overlaps(x, width)
                && previousBottom <= Math.max(platform.getY(),platform.getPreviousY()) + 0.001
                && nextY + height >= platform.getY()) {
            if(platform.getY()<landingY){landingY=platform.getY();landingPlatform=platform;}
        }
    }
    if (nextY + height >= landingY && previousBottom <= landingY + 0.001) {
        nextY = landingY - height;
        vy = 0;
        ground = true;
        jumpsUsed = 0;
        support=landingPlatform;
        if(support!=null)support.onLand();
    }
}
if (nextY < 0) { nextY = 0; vy = Math.max(0, vy); }
y = nextY;
```

โปรแกรมเพิ่มความเร็วแนวตั้ง vy ด้วยแรงโน้มถ่วงค่า 0.64 ในแต่ละรอบ แล้วคำนวณตำแหน่งถัดไป เมื่อผู้เล่นกำลังตก (vy >= 0) จะตรวจสอบแท่นที่ยังรองรับได้และมีตำแหน่งแนวนอนทับกับผู้เล่น โดยเปรียบเทียบตำแหน่งเท้าก่อนและหลังเคลื่อนที่ เพื่อป้องกันการตกทะลุแท่น เมื่อพบพื้นหรือแท่นที่ลงได้ จะปรับผู้เล่นให้ยืนบนผิว กำหนดความเร็วแนวตั้งเป็นศูนย์ และรีเซ็ตจำนวนครั้งที่กระโดด พร้อมเรียก onLand() เพื่อกระตุ้นพฤติกรรมของแท่น

3.Aim

```
public double worldX(int camera){
    return screenX+camera;
}
public double angle(double handX,double handY,int camera,boolean right){
    return active?Math.atan2(screenY-handY,worldX(camera)-handX):(right?0:Math.PI);
}
```

```
PlayerAimRig.Pose pose=PlayerAimRig.solve(player,mouseAim,camera);shotAngle=pose.angle;
shotX=pose.muzzleX(GUN_WIDTHS[selected]);shotY=pose.muzzleY(GUN_WIDTHS[selected]);
lastFireEffect=now;
for(int k=0;k<w.pellets;k++){
    double angle=shotAngle+(k-(w.pellets-1)/2.0)*.075;
    bullets.add(new Bullet(shotX,shotY,Math.cos(angle)*15,Math.sin(angle)*15,w.getDamage()));
}
lastFire=now;
```

โปรแกรมแปลงตำแหน่งเมาส์บนหน้าจอเป็นพิกัดในโลกเกมด้วยการบวกตำแหน่งกล้อง จากนั้นใช้ Math.atan2() คำนวณมุมจากตำแหน่งมือของผู้เล่นไปยังเมาส์ เพื่อนำไปกำหนดทิศทางอาวุธและกระสุน หากยังไม่มีการใช้งานเมาส์ จะใช้ทิศทางการหันของผู้เล่นเป็นมุมเริ่มต้น

4.ตรวจสอบการชน

```
private static boolean intersects(int x,int y,int w,int h,int xx,int yy,int ww,int hh){
    return x<xx+ww && x+w>xx && y<yy+hh && y+h>yy;
}
```

ตรวจสอบการชนโดยเปรียบเทียบขอบเขตสี่เหลี่ยมของวัตถุสองชิ้น หากขอบเขตซ้อนทับกันทั้งแนวนอนและแนวตั้ง เมธอดจะคืนค่า true เพื่อให้ระบบดำเนินการต่อ เช่น ลดพลังชีวิตเมื่อกระสุนชนศัตรู หากไม่ซ้อนทับกันจะคืนค่า false

5.พฤติกรรมบอส

```
if(--ticks>0)return;
switch(phase){
    case HUNT:
        targetX=p.getX()+p.getWidth()/2.0;targetY=p.getY()+p.getHeight()/2.0;
        phase=Phase.WARNING;ticks=isEnraged()?38:52;prepare();break;
    case WARNING: phase=Phase.ATTACK;ticks=level==7?100:60;attack();break;
    case ATTACK:phase=Phase.RECOVER;ticks=isEnraged()?35:55;break;
    case RECOVER:phase=Phase.HUNT;ticks=isEnraged()?35:65;break;
}
```

โปรแกรมแบ่งพฤติกรรมบอสเป็น 4 สถานะ ได้แก่ ไล่ล่าผู้เล่น (HUNT) เตรียมโจมตี (WARNING) โจมตี (ATTACK) และพักฟื้น (RECOVER) โดยใช้ตัวแปร ticks นับเวลาของแต่ละสถานะ เมื่อครบเวลาจะเปลี่ยนไปยังสถานะถัดไปและวนซ้ำ บอสจะบันทึกตำแหน่งเป้าหมายก่อนเตรียมโจมตี และเมื่อเข้าสู่สถานะคลั่งจะลดเวลาเตรียมโจมตีและพักฟื้น ทำให้ต่อสู้ได้รวดเร็วขึ้น

6.เลือกและปลดล็อกด่าน

```
rivate void select(int level){
    selected=level;
    boolean open=level<=unlocked;
    launch.setEnabled(open);
    launch.setText(open?"DEPLOY TO STAGE "+String.format("%02d",level)+"  >":"LOCKED  /  CLEAR STAGE "+(level-1));
    repaint();
}
```

```
ublic class Progress {
    private static final Preferences PREF=Preferences.userNodeForPackage(Progress.class);
    public static int unlocked(){
        return Math.max(1,Math.min(10,PREF.getInt("unlocked",1)));
    }
    public static void complete(int level){
        if(level>=unlocked())PREF.putInt("unlocked",Math.min(10,level+1));
    }
}
```

เมื่อผู้เล่นเลือกด่าน โปรแกรมจะเปรียบเทียบหมายเลขด่านกับระดับที่ปลดล็อก หากเข้าเงื่อนไขจะเปิดใช้งานปุ่ม DEPLOY หากด่านยังล็อกจะปิดปุ่มและแสดงด่านที่ต้องผ่านก่อน เมื่อผ่านด่านระบบเรียก complete() เพื่อบันทึกการปลดล็อกด่านถัดไปผ่าน Java Preferences โดยจำกัดไว้ไม่เกิน 10 ด่าน และเรียก unlocked() เพื่ออ่านความคืบหน้าเมื่อเปิดหน้าเลือกด่านอีกครั้ง

3.สรุปผล

ปัญหาที่พบ

1.การเคลื่อนไหวของตัวละครไม่เป็นธรรมชาติ

2.บอสดูเหมือนภาพเลื่อนและขาดความชัดเจนในการโจมตี

3.ไม่ทราบปริมาณเกราะที่เหลือ

4.ไม่มีเอฟเฟ็คการโจมตี

จุดเด่น

1.เกราะชีวภาพที่เปลี่ยนการป้องกันเป็นพลังโจมตี

เมื่อเกราะรับความเสียหายจะสะสมพลัง และเมื่อเต็มผู้เล่นสามารถปล่อยพลังโจมตีศัตรูรอบตัวได้ ทำให้เกราะมีบทบาททั้งป้องกันและสร้างโอกาสในการต่อสู้

2.บอสแต่ละด่านมีเอกลักษณ์ต่างกัน

ทั้ง 10 ด่านมีบอสที่มีรูปลักษณ์และรูปแบบการโจมตีแตกต่างกัน ผู้เล่นจึงต้องปรับวิธีรับมือแทนการใช้วิธีเดิมตลอดเกม

3.รูปลักษณ์เกราะเปลี่ยนตามพลังสะสม

เกราะชีวภาพมีสีดำอมม่วงและเส้นพลังเรืองแสง โดยแสดงผลตามระดับพลังสะสม ช่วยให้ผู้เล่นรับรู้สถานะผ่านตัวละครควบคู่กับแถบสถานะ

คำแนะนำสำหรับผู้สอนที่อยากให้อธิบาย

อยากให้มีคลิปเรียนย้อนหลังครับบางทีตามเนื้อหาไม่ทันในห้องแล้วไปศึกษาเองบางที่อาจจะไม่ตรงจุดกับที่อาจารย์สอนครับ